function2 remains as byte-code; only the returned object stored in the variable *g* is native-compiled.

```
psyco.bind(function1)
g = psyco.proxy(function2)
g(args)             #optimised
function2(args)     #unoptimised
function1(args)     #optimised
```

Psyco lets you boost your app's runtime performance with the addition of only a few lines of code, and a little understanding about the nature of your Python program and its various functions and operations. You can choose the speed-up mode that's most suitable for your Python program based on the broad rules mentioned above, and with a little trial and error. Psyco comes with many Python test programs. You can go through the documentation provided on the Psyco project page to explore it further.

Unladen Swallow, Google's steroids for CPython

The founder of Python, Guido van Rossum, works for Google. Python is one of the three official programming languages used there, along with C++ and Java—so who could be more concerned about the speed of Python programs? The result is Unladen Swallow, an optimisation branch of the standard Python code base. The main goal of Unladen Swallow is to maintain full compatibility with CPython and the libraries, while boosting performance around 5 times. It is based on Python 2.6.1, and changes only the Python run time, nothing else. Currently, *youtube.com* is making heavy use of the Unladen Swallow.

The Unladen Swallow project is addressing improvements in all the areas that make a Python program slow, as we discussed earlier. It has implemented JIT compilation to boost the runtime performance of Python programs. The runtime's current implementation converts the Python byte-code of all the hot functions first to LLVM (Low-Level Virtual Machine) IR (Intermediate Representation), and then those are finally compiled to native machine code. The long-term plans of the project are to change the stack-based CPython VM to a register-based VM, based on LLVM code, to extract more acceleration through the LLVM JIT on this VM. It also has put out many improvements to the CPython garbage collector, which lower the delays caused by the GC.

Unladen Swallow is an ongoing project. To use it, you have to fetch its code from the project SVN trunk, and build it. The prerequisites for building it are LLVM and Clang; you also need the standard build tools that we installed in the Psyco section, to build LLVM and Clang. The project page provides detailed information on installing it, so please follow that to set it up on your machine. The project page also provides the Python test suite to benchmark Swallow's performance against CPython's. You can also try to run your own Python programs with the Unladen Swallow Python interpreter, to judge the performance gains. Explore more about Unladen Swallow through the various links and documents provided on its project page.

Shedskin, a Python-to-C++ compiler

Shedskin pumps up the performance of existing Python code through a different approach. It takes Python programs written in a static subset of the standard Python language, and generates corresponding optimised C++ code. This generated C++ code can be compiled as a native-code application, or as an extension module to be imported in a larger Python program. Thus, Shedskin can boost your CPU-intensive program's speed. Currently, Shedskin is in an experimental stage, and supports only around 20 modules from the standard library. Still, it's a very promising project to use to turbo-charge your Python programs.

You can find *.deb* and *.rpm* packages of Shedskin on its project page, in addition to the source tarball. To install Shedskin from source, the prerequisites are g++, the Boehm garbage collector, and the PCRE library. You can install these on an Ubuntu machine by running *sudo apt-get install g++ libgc-dev libpcre3-dev*. You could build the GC and *libpcre* on platforms where no pre-built packages are available: download their tarballs from the links in the *Reference* section, and follow the provided build instructions.

Once the prerequisites are installed, download the latest tarball of Shedskin from its project page, and run *tar zxvf shedskin-version.tgz && cd shedskin-version*. Finally, run *python setup.py install* (with *sudo* or *su* privileges if you want to do a system-wide install). If everything goes well, check that running *shedskin* in the terminal shows some info.

Shedskin translates implicit Python types to explicit C++ types. It uses type-inference techniques to determine the types used in Python programs. It does iterative analysis of the Python code, and sees what values are assigned to different objects, to deduce their static types. Therefore, it doesn't allow you to assign different types to the variable (re-using the variable name with different data types is forbidden). It also forces you to use only similar types as the elements of various Python collections (tuple, list, set, etc). Here are a few examples of valid and invalid Python expressions in Shedskin:

```
i    = 56                          # valid
i    = 'a'                          # invalid
t1  = (1, 7, 9, 16)               # valid
t2 = (1, 's', 3.14, [1, 2, 3]) # invalid
t3 = ([1], [1, 6], [4, 7, 5])  # valid
a  = ['abc', 'a', 'efgh']         # valid
b  = [(0.01), (0.1, 3.178)]  # valid
c  = [1, 6.78, 5, 0]              # invalid
d1= {'1':'shed', '2':'skin'} # valid
d2= {'a':1, 2:5, 'c':7}         # invalid
```

The basic usage of Shedskin is *shedskin file.py*. It creates *file.hpp*, *file.cpp* and *Makefile* in the current working directory. It also provides a few switches to control the way output files are generated. For example, *-a* causes an additional annotated